Phase 1 Implementation Spec: Day 1-5 (English)

> **Conversational ERP Foundation Week** Goal: natural language "Order 1000 M6 bolts from ZhongGang for me" → AI asks for missing slots → Confirm Card → DB creates PO → can undo

Prerequisite: `CONVERSATIONAL_ERP_DESIGN_EN.md` (required reading) **Version**: v1.0 (2026-05-15)

📑 Contents

---

1. Phase 1 Goal

By end of Day 5, this script must be demonstrable to the owner:

```
[Open erpilot desktop UI, login as admin]
[Click "AI Assistant" on left sidebar]

User types: "Order 1000 M6 bolts from ZhongGang for me"

[AI returns inline Card in 2 seconds:
  📋 Confirm Purchase Order
  ──────────────────────
  Supplier:  ZhongGang (S-001)
  Part:      M6-BOLT-20 stainless steel bolt 20mm
  Qty:       1000
  Unit price: NT$ 0.5 (last transaction price)
  Total:     NT$ 500
  ──────────────────────
  [✓ Confirm]  [✏ Adjust]  [✗ Cancel]
]

[User clicks "Confirm"]

[AI: ✅ Created PO-2026-105. ETA 7 days.
     To cancel, say "undo last action" within 5 min.
     View → /purchase/orders/PO-2026-105
]

[User says: "undo last action"]

[AI: ✅ Reverted PO-2026-105. DB state restored to before creation.]

[User opens browser at /purchase/orders, PO-105 indeed gone]
```

Success = all 7 [...] steps above run end-to-end. Failing any step = Phase 1 incomplete.

Day 1: Tool Registry + Risk-Tier

Goal

Build unified tool registration mechanism. Each tool carries `risk_tier` / `slots` / `required_permission` metadata. Refactor existing 26 tools into this registry.

Deliverables

File	Action	Purpose
`backend/app/agents/registry.py`	**NEW**	`@register_tool` decorator + ToolMeta dataclass + query API
`backend/app/agents/__init__.py`	MODIFY	export registry
`backend/app/agents/domains/*_tools.py`	MODIFY × 10	add `@register_tool` to each tool
`backend/app/agents/engine.py`	MODIFY	fetch tool via registry instead of hardcoded list
`backend/tests/smoke/test_tool_registry.py`	**NEW**	verify all tools registered + risk_tier valid

`registry.py` API Contract

```python
from dataclasses import dataclass, field
from typing import Literal, Callable, Awaitable, Any
from enum import Enum

class RiskTier(str, Enum):
    READ = "read"
    SOFT_WRITE = "soft-write"
    HARD_WRITE = "hard-write"

@dataclass
class ToolMeta:
    name: str                           # e.g. create_purchase_order
    domain: str                         # purchase
    risk_tier: RiskTier
    description: str                    # for LLM (natural language)
    slots: list[dict] = field(default_factory=list)
    # slot = { name: "qty", type: "int", required: True, description: "..." }
    required_permission: str | None = None # RBAC permission code
    func: Callable[..., Awaitable[Any]] = None  # the actual function
    undo_recipe: Callable | None = None    # for soft/hard-write

# Global registry
_REGISTRY: dict[str, ToolMeta] = {}

def register_tool(
    name: str,
    domain: str,
    risk_tier: RiskTier,
    description: str,
    slots: list[dict] = None,
    required_permission: str | None = None,
    undo_recipe: Callable | None = None,
):
    def decorator(func):
        _REGISTRY[name] = ToolMeta(
            name=name, domain=domain, risk_tier=risk_tier,
            description=description, slots=slots or [],
            required_permission=required_permission,
            func=func, undo_recipe=undo_recipe,
        )
        return func
    return decorator

def get_tool(name: str) -> ToolMeta | None:
    return _REGISTRY.get(name)

def list_tools(domain: str | None = None, tier: RiskTier | None = None) -> list[ToolMeta]:
    """For AI engine to list (filtered) available tools."""
    items = list(_REGISTRY.values())
    if domain:
        items = [t for t in items if t.domain == domain]
    if tier:
```

```
        items = [t for t in items if t.risk_tier == tier]
    return items
```

Refactor Example: existing inventory_tools.py

```python
# Before
async def _query_inventory(db, user, part_no: str = None, ...): ...

# After
from app.agents.registry import register_tool, RiskTier

@register_tool(
    name="query_inventory",
    domain="inventory",
    risk_tier=RiskTier.READ,
    description="Query a part's inventory quantity. Provide part_no or part_id.",
    slots=[
        {"name": "part_no", "type": "str", "required": False,
         "description": "Part number, e.g. M6-BOLT-20"},
        {"name": "part_id", "type": "str", "required": False,
         "description": "UUID-form part id"},
    ],
    required_permission="inventory.part.read",
)
async def _query_inventory(db, user, part_no: str = None, part_id: str = None):
    ...
```

Day 1 Acceptance

- [ ] `app/agents/registry.py` exists + all 26 tools registered via decorator
- [ ] `engine.py` fetches tools via `get_tool()` / `list_tools()`, no hardcoded list
- [ ] `python -c "from app.agents.registry import list_tools; print(len(list_tools()))"` prints 26
- [ ] `pytest tests/smoke/test_tool_registry.py` all green (10 cases: each domain has ≥1 tool / all hard-write must have required_permission / ...)
- [ ] `bash scripts/run_gates.sh` 8/8 green

Day 2: ConfirmCard Schema + Frontend

Goal

Define the JSON schema returned by hard-write tools, and implement inline `<ConfirmCard>` React component in Chat page.

Deliverables

File	Action	Purpose
`backend/app/agents/confirm_card.py`	**NEW**	Pydantic schema for ConfirmCardResponse
`backend/app/api/chat.py`	MODIFY	Route hard-write to return ConfirmCard not execute
`frontend-desktop/src/components/ConfirmCard.tsx`	**NEW**	React component
`frontend-desktop/src/components/ChatMessage.tsx`	MODIFY	Detect type=confirm_required → render Card
`frontend-desktop/src/pages/Chat.tsx`	MODIFY	Confirm button → second API call with `confirm_token`
`backend/tests/smoke/test_confirm_card.py`	**NEW**	Schema validation + flow test

Schema Definition

```python
# backend/app/agents/confirm_card.py
from pydantic import BaseModel
from typing import Literal, Any
from datetime import datetime

class ConfirmCardResponse(BaseModel):
    type: Literal["confirm_required"] = "confirm_required"
    confirm_token: str            # backend-issued UUID, frontend echoes back
    summary_zh: str               # Chinese summary
    summary_en: str               # English summary
    action: str                   # tool name to execute
    args: dict[str, Any]          # tool arguments
    risk_tier: str                # "hard-write"
    undo_eligible: bool = True
    buttons: list[str] = ["confirm", "adjust", "cancel"]
    expires_at: datetime          # expires in 5 minutes
    explanation: str | None = None  # AI's additional explanation
```

Frontend Component Interface

```tsx
// frontend-desktop/src/components/ConfirmCard.tsx
interface ConfirmCardProps {
  confirm_token: string
  summary: string            // i18n-resolved version
  action: string
  args: Record<string, any>
  expires_at: Date
  on_confirm: () => void
  on_adjust: () => void
  on_cancel: () => void
}

export function ConfirmCard(props: ConfirmCardProps) {
  // Visual: yellow border + icon + summary + 3 buttons + countdown timer
}
```

Day 2 Acceptance

- [ ] `pytest tests/smoke/test_confirm_card.py` all green
- [ ] In frontend Chat page, with a hardcoded dummy hard-write response, can render the Card (3 buttons + countdown)
- [ ] Click "Cancel" → Card disappears, chat log shows "Cancelled"
- [ ] Click "Confirm" → shows "Executing..." state (Day 3 will wire real execution)
- [ ] `bash scripts/run_gates.sh` 8/8 green

Day 3: First Hard-Write Tool — `create_purchase_order_with_confirm`

Goal

Truly connect: natural language "create a PO" → ConfirmCard → confirm → DB really has the PO.

Deliverables

File	Action	Purpose
`backend/app/agents/domains/purchase_write_tools.py`	NEW	`create_purchase_order` write tool
`backend/app/agents/engine.py`	MODIFY	Handle confirm_token: second call actually executes
`backend/tests/integration/test_create_po_via_chat.py`	NEW	End-to-end integration test

Tool Skeleton

```python
@register_tool(
    name="create_purchase_order",
    domain="purchase",
    risk_tier=RiskTier.HARD_WRITE,
    description="Create a purchase order. Requires supplier and item list.",
    slots=[
        {"name": "supplier", "type": "str", "required": True,
         "description": "Supplier name or code"},
        {"name": "items", "type": "list", "required": True,
         "description": "Items: [{part_no, qty, unit_price?}]"},
    ],
    required_permission="purchase.order.create",
    undo_recipe="cancel_purchase_order",
)
async def create_purchase_order(
    db, user, *,
    supplier: str, items: list[dict],
    confirmed: bool = False, confirm_token: str | None = None,
):
    # 1. Resolve supplier (entity-resolver)
    s_candidates = await search_supplier(db, supplier)
    if len(s_candidates) > 1:
        return DisambiguationResponse(...)
    if not s_candidates:
        return {"error": f"Supplier '{supplier}' not found"}
    supplier_obj = s_candidates[0]

    # 2. Resolve items
    resolved_items = []
    for it in items:
        part = await get_part_by_no(db, it["part_no"])
        if not part:
            return {"error": f"Part '{it['part_no']}' not found"}
        # Missing unit_price → fetch last
        if "unit_price" not in it:
            it["unit_price"] = await last_supplier_price(db, supplier_obj.id, part.id) or
part.unit_cost
        resolved_items.append({**it, "part_id": part.id})

    total = sum(it["qty"] * it["unit_price"] for it in resolved_items)

    # 3. Not yet confirmed → return ConfirmCard
    if not confirmed:
        token = await issue_confirm_token(
            user_id=user.user_id,
            action="create_purchase_order",
            args={"supplier_id": supplier_obj.id, "items": resolved_items},
        )
        return ConfirmCardResponse(
            confirm_token=token,
            summary_zh=f"建立 PO 給 {supplier_obj.name} · "
                       f"{len(resolved_items)} 項 · NT$ {total:.0f}",
```

```
            summary_en=f"Create PO to {supplier_obj.name} · "
                       f"{len(resolved_items)} items · NT$ {total:.0f}",
            action="create_purchase_order",
            args={"supplier_id": supplier_obj.id, "items": resolved_items},
            risk_tier="hard-write",
            expires_at=now() + timedelta(minutes=5),
        ).dict()

    # 4. Confirmed → actually execute
    await validate_confirm_token(token=confirm_token, user_id=user.user_id)
    po = await purchase_service.create_purchase_order(
        db, {"supplier_id": supplier_obj.id, "items": resolved_items},
        user=user.raw_user,
    )
    # 5. Write ActionHistory (for undo)
    await action_history.write(
        session_id=user.session_id, user_id=user.user_id,
        tool="create_purchase_order",
        args_after={"po_id": po.id, "po_no": po.po_no},
        undo_recipe=f"cancel_purchase_order(po_id='{po.id}')",
        expires_minutes=5,
    )
    return {
        "ok": True,
        "po_no": po.po_no,
        "po_id": po.id,
        "message": f"Created PO {po.po_no}",
    }
```

Day 3 Acceptance

- [ ] Integration test: 2 messages via `/api/chat-v2`:
  - 1st: "Order 1000 M6-BOLT-20 from ZhongGang" → returns ConfirmCard JSON
  - 2nd: with `confirm_token` → DB has new PO
- [ ] Integration test: 2nd call with expired token → 410 Gone
- [ ] User without permission → "You don't have purchase.order.create permission"
- [ ] `bash scripts/run_gates.sh` 8/8 green

Day 4: Slot-filling Reverse-Ask

Goal

AI proactively asks when slots are missing. Users **don't need to say everything in one sentence**.

Deliverables

File	Action	Purpose
`backend/app/agents/slot_filling.py`	NEW	Find missing slots + generate ask-back prompt
`backend/app/agents/engine.py`	MODIFY	Run slot-filling check before tool call
`backend/app/models/ai_governance.py`	MODIFY	DecisionLog adds `slots_state` JSON field
`backend/tests/integration/test_slot_filling.py`	NEW	5 scenarios of reverse-ask

Reverse-Ask Strategy

LLM prompt template:

```
You are erpilot's purchase assistant. User wants to create a PO but is missing info:
- Missing: {missing_slots}
- Known: {filled_slots}

Ask back in one Chinese sentence (< 30 chars), friendly and specific.
If multiple missing, ask the most important one first.
No explanations or "please tell me" boilerplate.
```

Day 4 Acceptance

- [ ] Integration test 5 scenarios:
  1. "Create a PO" → AI asks "Which supplier?"
  2. "PO to ZhongGang" → AI asks "What part number?"
  3. "PO M6 to ZhongGang" → AI asks "How many?"
  4. "PO 1000 M6 to ZhongGang" → AI enters disambiguation (3 M6 candidates)
  5. "PO 1000 M6-BOLT-20 to ZhongGang" → AI returns ConfirmCard directly
- [ ] `bash scripts/run_gates.sh` 8/8 green

Day 5: E2E Demo + Recording

Goal

String together the full script, record 30-60 sec video, store in `docs/demos/` for marketing.

Work Items

Task	Detail
Final integration test	1 big test running through §1's 7-step script
Demo script	`scripts/demo/conversational_po.md` for users to follow
Screen recording	OBS or Windows Game Bar, 30-60 sec
Video file	`docs/demos/phase1_conversational_po.mp4` (or .gif)
WORKLOG update	Document Phase 1 outcomes
README demo link	"Conversational ERP demo" featured prominently

Demo Script (read/follow during recording)

```
0:00 [Open browser, http://localhost:5173]
0:03 [Login admin / admin123]
0:05 [Click AI Assistant]
0:08 Voice: "erpilot doesn't need training — boss just speaks."
0:12 [Type: Create PO to ZhongGang]
0:15 [AI: "What part number?"]
0:17 [Type: 1000 M6]
0:20 [AI Disambiguation: M6-BOLT-20? M6-NUT? M6-WASHER?]
0:25 [Click M6-BOLT-20]
0:28 [AI ConfirmCard: amount NT$500, 3 buttons]
0:32 Voice: "Medium-risk actions require confirmation — never accidental."
0:36 [Click "Confirm"]
0:40 [AI: ✅ Created PO-2026-105]
0:42 Voice: "Within 5 minutes you can undo."
0:45 [Type: undo last action]
0:48 [AI: ✅ Reverted]
0:52 [Switch to browser /purchase/orders, PO indeed gone]
0:58 Voice: "Natural language ERP. 2-hour training. erpilot."
1:00 [Logo + Tagline]
```

Day 5 Acceptance

- [ ] Video produced (30-60 sec, including §1's 7-step script)
- [ ] `tests/integration/test_e2e_conversational_po.py` — one big test running through all 7 steps, all green
- [ ] WORKLOG #17 documents Phase 1 outcomes in detail
- [ ] CLAUDE.md bumped to v2.9
- [ ] README adds demo section (gif or video embed)
- [ ] commit + push + CI green

7. Cross-Day Test Plan

Each Day must run:

```
# Existing 138 tests + Day N's new tests
cd backend && python -m pytest tests/ -q --tb=line

# Full gate run
bash scripts/run_gates.sh    # must be 8/8 green
```

No regression allowed.

New test matrix (cumulative over 5 days):

Day	New tests	Cumulative
1	tool_registry × 10	148
2	confirm_card × 8	156
3	create_po_via_chat × 12	168
4	slot_filling × 5	173
5	e2e_conversational_po × 1 (big test)	174

8. Definition of Done

At Phase 1 end, **all** must be checked:

- [ ] §1's 7-step script runs e2e (pytest)
- [ ] A 30-60 sec demo video committed to git
- [ ] `bash scripts/run_gates.sh` all green
- [ ] CI green
- [ ] WORKLOG #17 + CLAUDE.md v2.9 pushed
- [ ] 138 → 174 tests, 0 regressions
- [ ] Phase 2 can extend linearly (registry framework verified, each subsequent tool is half-day)

Anything unchecked = Phase 1 not done. Do NOT proceed to Phase 2.

Related Documents

- Architecture Design (required prereq)
- **Chinese version**: `CONVERSATIONAL_ERP_PHASE1_SPEC_ZH.md`

Version: v1.0 · **Last updated**: 2026-05-15 · **Status**: Phase 1 launch spec